

Known issues and divergences

This file records, in public, where this code and the article line up and where they do not. It exists so that a reader who runs the code is not surprised.

`notebooks/` holds the runs that produced the reported results. `src/fedrest/` is a cleaned reference implementation of the same mechanisms. `scripts/verify_against_paper.py` checks that reference implementation against every privacy and communication figure printed in the article — 24 assertions, no dataset and no GPU required.

1. Placement of the Gaussian noise — agrees with the article

An earlier draft of this file claimed the notebooks add noise once per round to the parameter vector. **That was wrong and the claim is withdrawn.**

`LocalRenyiDPEngine.clip_and_add_noise` operates on `p.grad`: it clips the per-batch gradient to `max_grad_norm` and perturbs that gradient, once per optimisation step, inside the training loop. That is Eq. (2) of the article. `src/fedrest/training.py` implements the same structure, so the reference library and the notebooks agree here.

2. The noise-scale expression varies between notebooks

The notebooks do not all compute the standard deviation the same way:

Expression	Used in
$\frac{\text{sigma} = C * \sqrt{\alpha / (2 * \text{eps})}}{\text{— Eq. (4)}}$	<code>VERIFY_WUSTL_RR_eps0.5_alpha5.0</code> , <code>HIDS_FEDSPLITFTIL_RRC_SCVIC2021</code> , <code>HIDS_IIoT_RR_ToNIoT_Network_Enhanced</code>
$\frac{\sqrt{2a \cdot \ln(1/d)/e^2} \cdot \sqrt{T \cdot \ln(1/d))) \cdot \text{factor}}$	<code>HIDS_FL_LRDP_RR_IIoT_WUSTL_..._with_poisoning</code> , <code>HIDS_FEDREST_LRDP_RR_TONIOTPROCESS_with_poisoning</code>
fixed <code>noise_multiplier</code>	<code>HIDS_Fedrest_RR_ToNIoT_Linux_Process</code> , <code>Copy_of_HIDS_FEDSPLITFTIL_RRC_WUSTL</code>

Only the first is the Rényi calibration the article states. The second folds a composition term into the per-round scale, where Rényi composition is applied once afterwards (Eq. 5); the third is not calibrated to a stated `eps` at all.

`src/fedrest/privacy.py` is the canonical version, and `src/fedrest/legacy.py` implements the alternative expression so the difference is explicit rather than hidden. `scripts/audit_configs.py` prints each notebook's declared configuration.

This is a code-consistency observation, not a claim that any result is wrong. It means a reader reproducing a given table should use the notebook named for it in `(notebooks/MANIFEST.md)` rather than assuming all notebooks share one privacy implementation.

3. WUSTL-IIoT-2021 headline — verified

The WUSTL result was re-run at the configuration the article states, with `(sigma)` from Eq. (4).

Notebook: `(notebooks/VERIFY_WUSTL_RR_eps0.5_alpha5.0.ipynb)` Output:

`(results/VERIFY_WUSTL_RR_eps0.5_alpha5.0.txt)`

```
Privacy Budget (e):    0.5      Renyi Order (a):    5.0
Gradient Clipping:    1.0      Noise Scale:        1.0    ->  sigma = 2.2361
Rounds Completed:     50/50
Accuracy:              0.9830   Balanced Accuracy: 0.9889
F1 (Weighted):         0.9837   Best Validation F1: 0.9943 (round 30)
```

The article reports exactly these figures. An earlier version of the manuscript reported 99.77% accuracy; that figure was not reproduced and has been corrected throughout, together with Figure 2, which is now generated from this run's per-round log by `(scripts/make_figure2.py)`.

Two observations worth recording:

- An earlier run of the same experiment used `(sigma = 10.83)` and reached 98.31%. This run used `(sigma = 2.2361)` — 4.8x less noise — and reached 98.30%. The model is insensitive to the noise scale over this range.
- Balanced accuracy under DP (98.89%) is **higher** than without it (96.50%), and attack-class recall reaches 0.9959. The privacy mechanism costs accuracy on the benign majority and gains on the minority class.

4. `(delta_eps_trust = 0)` is only partly justified

Section 3.2.3 argues by post-processing that FedTrust costs no privacy. That holds for two of its three components:

- `(T_beh)` — a function of already-noised parameters. Post-processing ✓
- `(T_res)` — device metadata, non-sensitive under the threat model ✓
- `(T_temp)` — uses F1 and AUC computed on a **held-out slice of raw local data**. Post-processing covers functions of a mechanism's *output*; raw data is an *input* here, so releasing this score is a fresh query on private data and is not free ✗

The trust score is sent to the server every round, so under the honest-but-curious server model this is an unaccounted channel.

5. A synthetic data generator is present in one notebook

`HIDS_IIoT_RR_ToNIoT_Network_Enhanced.ipynb` defines `create_synthetic_toniot_data()`, labelled "*For demonstration*", with a real loader and preprocessing path above it.

No reported result comes from synthetic data. The function is retained for provenance. Do not call it.

6. Seed reporting

Table 2 fixes `random seed = 42`. Tables 6, 8 and 9 report mean $\pm$ standard deviation over three seeds. The headline results are single runs at seed 42 and carry no error bars.

7. The federation is simulated

Clients execute sequentially in one process; there is no network between them. No wall-clock latency or energy figure is reported anywhere, because any such number would come from a device power model rather than from measurement. The communication *volume* analysed in Section 6.5 is unaffected, being a property of the architecture rather than of the platform.

8. Notebook duplication

Four notebooks exist in near-duplicate `Copy_of_` form with small differences. Both members of each pair are kept rather than guessed between. `notebooks/MANIFEST.md` records what is known about each.